

Hierarchical Clustering

Prerequisite

Student should know about:

- Partition based clustering algorithm, *K-Means*
- Basic of graph theory

Learning Objective

After completing this chapter, students should be able to:

- Explain the concepts of *Proximity Matrix*, *Nested Partition*, *Denogram*
- Differentiate between *Hierarchical Clustering* and *Partition Based Clustering*
- Identify *Agglomerative* vs *Divisive* Clustering
- Explain and implement *DIANA* algorithm

Introduction

In the previous chapter, you have learned K-Means and its variants. In K-Means algorithm, you had to explicitly specify the number of clusters required. This can sometimes be difficult because we may not exactly know the number of clusters to begin with. To this rescue comes the hierarchical clustering. In hierarchical clustering, you do not need to specify the number of clusters because it does not generate a specific number of clusters rather it generates hierarchy of clusters. Furthermore, the final result of K-Means depends on the initialization of the centroids which in most cases is a random process; hence, making the final result random. But, most of the time the result in hierarchical clustering is deterministic which is required in many applications.

In this chapter, you are going to learn about the hierarchical clustering and understand how it solves the problems of partition based clustering like *K-Means*. First off, let's learn some of the key terminologies that are important in hierarchical clustering.

Proximity Matrix

Let us assume that we have following data points $D = \{x_1, x_2, x_3, x_4\}$. The proximity matrix given below tells us about the dissimilarity or distance between two data points.

$$D = \begin{bmatrix} 0 & 0.2 & 1.0 & 2 \\ 0.2 & 0 & 0.6 & 0.1 \\ 0.1 & 0.2 & 0 & 0.5 \\ 2 & 0.1 & 0.5 & 0 \end{bmatrix}$$

The distance matrix $\mathcal{D}$ for four data points is of size 4×4 . Also, $\mathcal{D}_{i,j}$ denotes the distance between x_i and x_j given as,

$$\mathcal{D}_{i,j} = d(x_i, x_j)$$

If $d(x_i, x_j) = d(x_j, x_i) \quad \forall_{i,j}$ (changing order of data points), then the distance matrix would be a symmetric matrix.

Also, the proximity matrix can have either the distance or the similarity between the data points. The similarity measure for example can be cosine similarity and the distance measure can be euclidian distance.

Finding the correct proximity metric for the problem at hand can be challenging and there are no shortcuts in doing so. In most of the cases, the domain knowledge is used to find an optimal proximity metric. In this lesson, we will mainly focus on the euclidian distance.

In the hierarchical clustering, only the proximity matrix is required for making the hierarchy of clusters; hence, we do not actually need to know how the data are distributed in the space. This can be helpful in many real world applications.

Nested Partition

Before moving forward, let's understand about nested partition and why we need it. The partition or cluster in general are represented with a set. Let's imagine we have the following dataset,

$$\mathcal{D} = \{x_1, \dots, x_n\}$$

Now, a partition $\mathcal{A}$ splits the data into the subsets $\{\mathcal{A}_1, \dots, \mathcal{A}_m\}$ that satisfies the following, $\mathcal{A}_i \cap \mathcal{A}_j = \emptyset \quad \forall_{i,j} \quad i \neq j$

Where $\cap$ represents intersection and $\emptyset$ is empty set. This equation means data points should belong to only one cluster.

$$\mathcal{A}_1 \cup \dots \cup \mathcal{A}_m = \mathcal{D}$$

Here, $\cup$ represents the union set operation. This equation means by combining those disjoint sets $\mathcal{A}_1, \dots$, we can obtain the original set we began with.

Now, imagine a new partition $\mathcal{B}$ splits $\mathcal{A}$. For $\mathcal{B}$ to be nested in $\mathcal{A}$; we should be able to create the partitions of $\mathcal{A}$ by combining the partitions of $\mathcal{B}$. The combining operation refers to the union operation in this case.

EXAMPLE:

Let's say we have a sample dataset given as,

$$\mathcal{D} = \{x_1, x_2, x_3, x_4, x_5, x_6, x_7\}$$

Imagine the partition $\mathcal{A}$ splits it into,

$$\mathcal{A} = \{(x_1, x_2, x_5, x_7), (x_3, x_4, x_6)\}$$

As you can see, each partitions are disjoint and $\mathcal{A}$ contains all the element of the original set $\mathcal{D}$.

With the next partition operation $\mathcal{B}$ in $\mathcal{A}$, we get the following result,

$$\mathcal{B} = \{(x_1, x_5), (x_2, x_7), (x_3), (x_4, x_6)\}$$

To determine if $\mathcal{B}$ is a nested partition or not, we can try to combine the partitions in $\mathcal{B}$ in order to obtain $\mathcal{A}$.

In above example, partition (x_1, x_2, x_5, x_7) can be created by combining the first and second partition of $\mathcal{B}$. Similarly, partition (x_3, x_4, x_6) can be created by combining the third and fourth elements of $\mathcal{B}$; therefore, the partition $\mathcal{B}$ is a nested partition of $\mathcal{A}$.

Let's see an example where the new partition is not a nested partition. If we had splitted the partition $\mathcal{A}$ with $\mathcal{B}'$ as follows,

$$\mathcal{B}' = \{(x_1, x_3), (x_2, x_5, x_7), (x_4, x_6)\}$$

You can check that we would not be able to obtain our original partition $\mathcal{A}$ by merging the partitions of $\mathcal{B}'$ making $\mathcal{B}'$ not a nested partition.

Now, the nested partition comes in hierarchical clustering because the hierarchical clustering can be defined in terms of nested partition as follows:

"A hierarchical clustering is a sequence of nested partition where each subsequent partition is nested on the previous one."

So, in the above example, the sequence $[\mathcal{D}, \mathcal{A}, \mathcal{B}]$ is a hierarchical clustering, whereas $[\mathcal{D}, \mathcal{A}, \mathcal{B}']$ is not.

Dendrogram

A more convenient way of representing the hierarchical clustering is by using a tree like structure called *dendrogram*. An example of dendrogram is shown in the figure below:

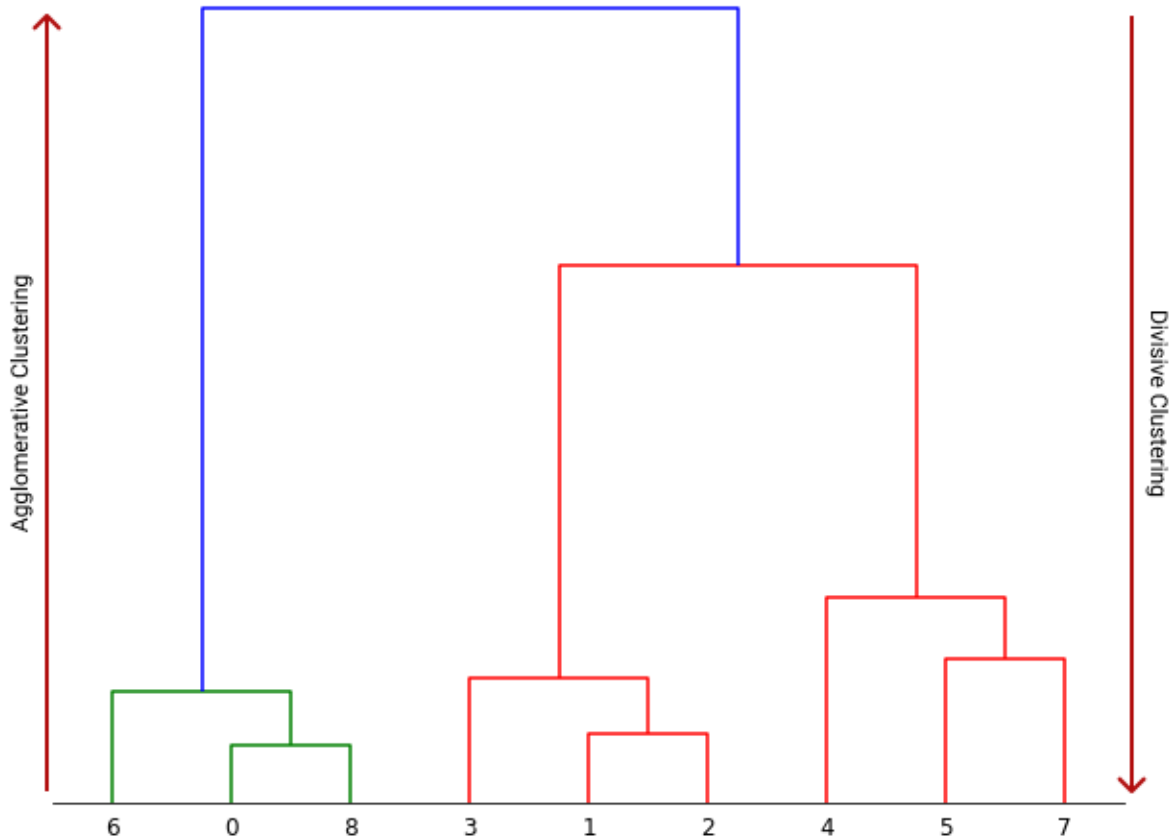


Figure 1: Dendrogram with arrow showing agglomerative and divisive clustering.

In the above Figure, you can see that it is a binary tree representation where each node splits into two branches. Using the terminology of tree, we can say that the root node here represents the partition containing all the data points, each node represents a partition and finally the leaf nodes are singleton partition (partition containing only one element). Similarly, the height of each edge is proportional to the distance between two data points, two partitions or a data point and a partition.

In dendrogram, the root node is just our dataset and the leaf node is just the data points, so they are of no use. Now, to get some required clusters in hierarchical clustering, we can cut the dendrogram at any level shown by the dotted line parallel to x -axis.

One of the advantages of dendrogram is that it gives an intuitive understanding of relationship between the clusters that can be very helpful in some application like taxonomy.

Now let's create a dendrogram using a sample data set .

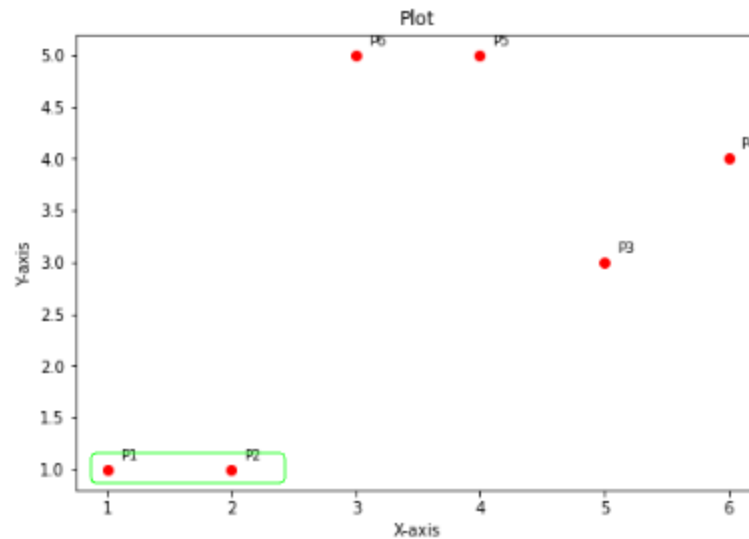


Figure 2: Scatter Plot of data

Here, each data point is a cluster of its own. We want to determine a way to compute the distance between each of these points. For this, we try to find the shortest distance between any two data points to form a cluster. Once we find those with the least distance between them, we start grouping them together and forming clusters of multiple points.

This can be represented in dendrogram as shown below.

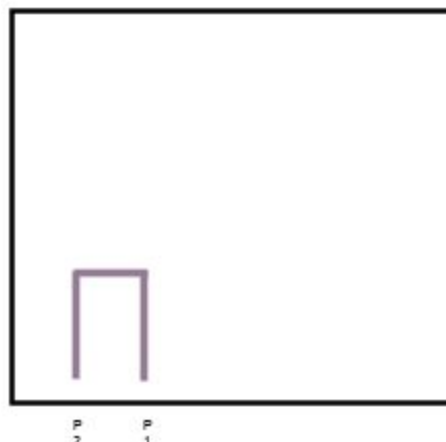


Figure 3: First Dendrograph

Here we will have 3 groups: P1-P2, P3-P4, and P5-P6 their respective dendrograms can be constructed as below.

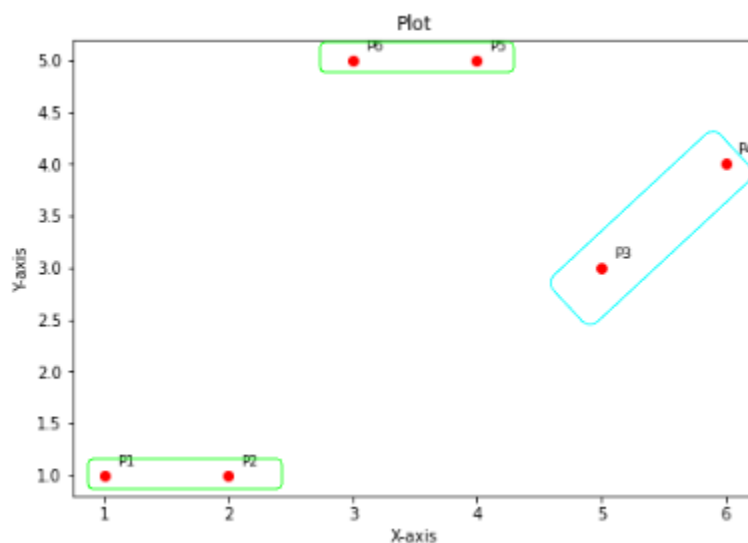


Figure 4: Plot

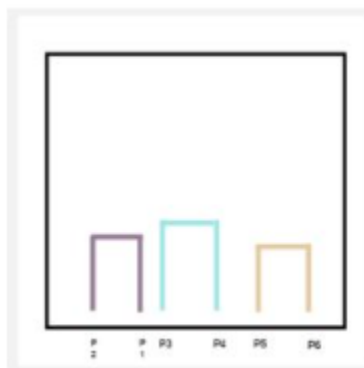


Figure 5: Dendrograph

</div>

Now we will bring two close groups together. P3-P4 and P5-P6 are all under one dendrogram because they are together than P1-P2 group.

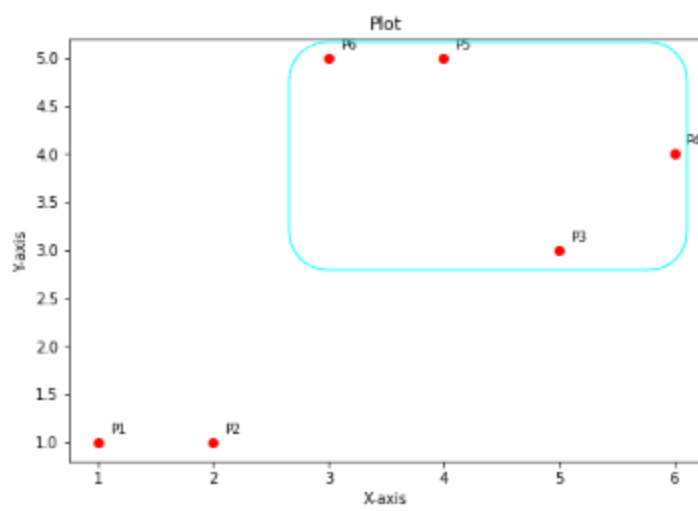


Figure 6: Plot

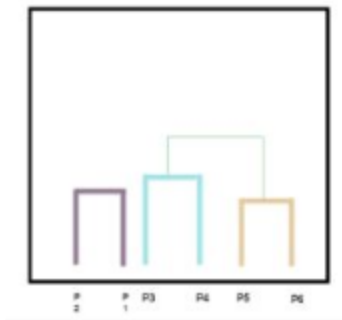


Figure 7: Dendrogram

Now we will bring another group of clusters together with the big cluster. If we have multiple other clusters we merge another cluster to this one according to the shortest distance.

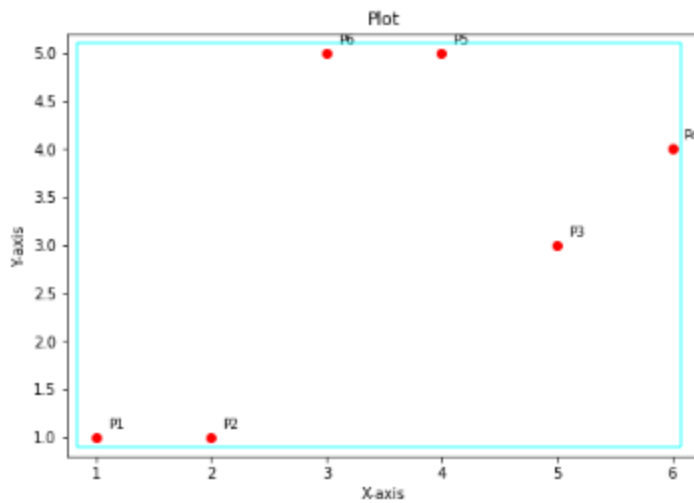


Figure 8: Plot

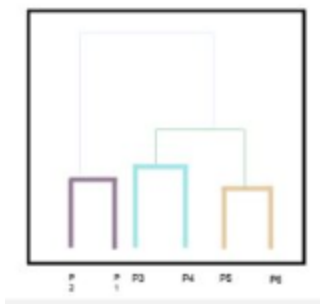


Figure 9: Dendrogram

In this way, we can find the dendrogram graph of the dataset.

Types of Clustering

Hierarchical clustering can be divided into two parts given in the list below:

- Agglomerative Clustering

Agglomerative clustering, also known as bottom-up hierarchical clustering, is a clustering approach where we start from the bottom of the hierarchy/dendrogram and build the partition up until a specified stopping condition. The upward arrow in Figure 1 shows the process of agglomerative clustering where we start from singleton partition and build up the tree until we reach the root node.

- Divisive Clustering

Divisive clustering is the opposite of agglomerative clustering in the sense that the dendrogram is created in top to bottom fashion. Initially all the datapoints belongs to a single cluster, then we iteratively apply the split until a stopping criteria is reached or all the clusters are singleton. The downward arrow in Figure 1 shows the process of divisive clustering.

In this chapter, you are going to learn about divisive clustering and in next chapter you will learn about agglomerative clustering.

Divisive Clustering

Divisive clustering algorithm is a top-down approach where we start at the top of the tree (root node) and create clusters by splitting a single cluster into two clusters.

The working of divisive hierarchical clustering algorithm is shown in the animation below.

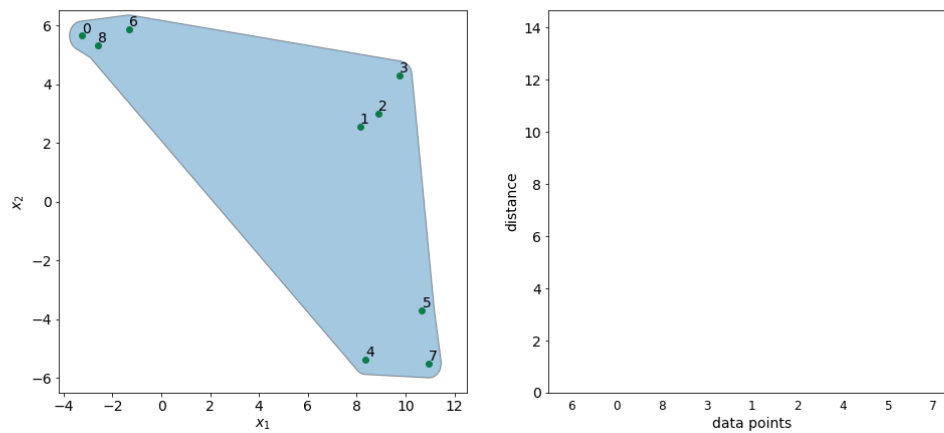


Figure 7: Divisive clustering on a toy dataset. (a) Scatter plot with highlighted clusters animation. (b) Divisive clustering shown in dendrogram animation

In the Figure 3(a), the circular boundary around the data represents a cluster. With divisive clustering, you can see in the scatter plot (figure (a)) that we start by assigning all the data points to a single cluster, then we iteratively split the cluster to form smaller clusters until we reach the stopping condition or we obtain all the singleton cluster. In the dendrogram plot in Figure 7(b), contrary to agglomerative clustering, we start from the root node of the binary tree and start the branch formation from the top to bottom fashion until we achieve all the leaf nodes as a singleton partitions.

General Algorithm

A general divisive hierarchical clustering algorithm is given below,

ALGORITHM: General Divisive Hierarchical Clustering

Start with root node containing all the data points

REPEAT: >

Split parent cluster C into two parts C_a and C_b using a split method

Choose the next cluster from non-singleton leaf clusters with maximum value of squared error.

UNTIL: All the leaves are singleton clusters

In the above algorithm, we talked about the splitting method but we did not go into the details. Also, we choose the next cluster using maximum squared error which is not the only way to do so. Here, we will discuss about some of the divisive method and criterion and also the method of choosing the next cluster for the split. >

- Split method and criterion

We should consider how are we going to split a given cluster into smaller ones. We can use Bisecting K -Means algorithm for this purpose. Also, if the data are categorical we can use

the Gini-index.

- Choosing the cluster to split

Choosing which cluster to split next can also be important when we are trying to achieve good results early on in the clustering.

We are going to cover DIANA divisive clustering algorithm in the following section in detail.

DIANA Algorithm

DIANA (Divisive ANALysis) is a heuristic method for divisive clustering.

Let's imagine we are trying to split the cluster $\mathcal{C}$ into two clusters $\mathcal{C}_a$ and $\mathcal{C}_b$. The DIANA algorithm for this purpose is given below:

ALGORITHM: Divisive ANALysis (DIANA)

1. Assign all data of $\mathcal{C}$ to $\mathcal{C}_a$ and $\mathcal{C}_b$ to an empty cluster.
2. For each data point in $\mathcal{C}_a$, i.e $\mathbf{x}_i \in \mathcal{C}_a$, do
 - a. Compute the average distance of x_i to all other data points as,

$$d(\mathbf{x}_i, \mathcal{C}_a \setminus \{\mathbf{x}_i\}) = \frac{1}{N_{\mathcal{C}_a} - 1} \sum_{\substack{\mathbf{x}_j \in \mathcal{C}_a \\ i \neq j}} d(\mathbf{x}_i, \mathbf{x}_j)$$

- b. For the remaining iteration, compute the difference between the average distance to $\mathcal{C}_a$ and $\mathcal{C}_b$ as,

$$D_i = d(\mathbf{x}_i, \mathcal{C}_a \setminus \{\mathbf{x}_i\}) - d(\mathbf{x}_i, \mathcal{C}_b) \quad (1)$$

$$= \frac{1}{N_{\mathcal{C}_a} - 1} \sum_{\substack{\mathbf{x}_j \in \mathcal{C}_a \\ i \neq j}} d(\mathbf{x}_i, \mathbf{x}_j) - \frac{1}{N_{\mathcal{C}_b}} \sum_{\mathbf{x}_k \in \mathcal{C}_b} d(\mathbf{x}_i, \mathbf{x}_k) \quad (2)$$

3. a. For first iteration, add the data point x_i with the maximum D_i to the empty cluster $\mathcal{C}_b$
 - b. For iterations except first

IF: maximum value of D is greater than zero given as,

$$D_{max} = \max(D_1, D_2, \dots) > 0$$

move the data point x_{max} corresponding to D_{max} to $\mathcal{C}_b$

Repeat Step 2(b) and 3(b)

OTHERWISE: stop

With this algorithm you will be able to cluster any given data points according to the divisive clustering approach.

Key Takeaways

- Proximity Matrix gives the nearness or distance between any two data points
- Hierarchical clustering gives multiple clusters arranged in hierarchy or in terms of nested partition, it gives a list of nested partitions
- Dendrogram can be very useful when visualizing and understanding the clustering
- Divisive divides a single big clusters into small ones from the top down fashion
- DIANA is one of the popular divisive algorithm where the division is done based on the closeness to two chosen points from the cluster

References

- **Algorithms for Clustering Data** by *Anil K. Jain and Richard C. Dubes*

Look into the chapter 3 : Clustering methods and algorithms

- A Comparative Study of Divisive and Agglomerative Hierarchical Clustering Algorithms by *Maurice Roux*

<https://hal.archives-ouvertes.fr/hal-02085844/document>